الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء السابع: مكتبة Plotly

الفصول ١ - ٦: من التعريف بالمكتبة حتى الدوال الأساسية

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة 4
الفصل الأول: التعريف بمكتبة Plotly 5
١.١ نشأتها والمشكلة التي حلّتها 5
١.٢ الاستخدامات والشركات 5
١.٣ المميزات والعيوب 5
١.٤ متى تستخدمها ومتى لا 5
١.٥ أشهر البدائل 5
ملخص الفصل 6
الفصل الثاني: التثبيت والإعداد 7
٢.١ أفضل أسلوب استيراد 7
٢.٢ بيئات العرض (Rendering) 7
أفضل الممارسات 7
الفصل الثالث: فلسفة التصميم 8
٣.١ واجهتان بفلسفتين مختلفتين 8
٣.٢ البنية الداخلية: Figure = Data + Layout 8
٣.٣ JSON كلغة وصف مشتركة 8
٣.٤ Templates: فلسفة الأنماط الموحدة 8
ملخص الفصل 9
الفصل الرابع: المفاهيم الأساسية 10
٤.١ Traces 10
٤.٢ Layout 10
٤.٣ التفاعلية المدمجة افتراضيًا 10
٤.٤ Subplots في Plotly 10
الأخطاء الشائعة 10
الفصل الخامس: الكائنات وواجهات البرمجة 11
٥.١ التوقيع القياسي لدوال Plotly Express 11
٥.٢ Method Chaining عبر *update_ 11
٥.٣ نموذج التنفيذ: بناء تصريحي ثم تصيير في المتصفح 11
٥.٤ التكامل مع Dash 11
أفضل الممارسات 11
الفصل السادس: أهم الدوال 13
٦.١ أهم دوال Plotly Express 13

٦.٢ الرسوم المتحركة عبر animation_frame 13
٦.٣ التصدير 13
٦.٤ مقارنة بين Plotly وMatplotlib وSeaborn 13
الأخطاء الشائعة 14
الأسئلة الشائعة 14
تمارين عملية 14
حلول مقترحة (تلميحات) 14
مراجع وروابط رسمية 14
خاتمة الجزء الأول من Plotly 15

مقدمة

بعد Matplotlib، ننتقل إلى Plotly: مكتبة رسم تفاعلية بالكامل، مبنية على مكتبة جافاسكريبت plotly.js، وتنتج رسومًا يمكن تكبيرها وتصفيتها والتحويم فوقها لعرض التفاصيل، دون أي كود جافاسكريبت من طرفك. هذا يجعلها الخيار الطبيعي لتطبيقات الويب ولوحات المعلومات الحية ومنصات مثل Dash.

الفصل الأول: التعريف بمكتبة Plotly

١.١ نشأتها والمشكلة التي حلّتها

أسّست شركة Plotly (سابقًا Plotly Technologies) المكتبة عام ٢٠١٢، بهدف سد فجوة واضحة: مكتبات الرسم التقليدية في بايثون مثل Matplotlib أنتجت صورًا ثابتة، بينما احتاج المحللون ومطوّرو الويب رسومًا تفاعلية غنية (Zoom, Hover, Pan, Filter) قابلة للتضمين مباشرة في صفحات الويب دون كتابة جافاسكريبت يدويًا. بُنيت Plotly للغة بايثون كواجهة تولّد كود plotly.js تلقائيًا خلف الكواليس، فيحصل المستخدم على قوة الويب التفاعلية بصياغة بايثون خالصة.

١.٢ الاستخدامات والشركات

- لوحات معلومات تفاعلية حية عبر إطار عمل Dash المبني فوق Plotly.
- تقارير بيانات مالية واقتصادية تفاعلية (تكبير على فترات زمنية معينة).
- استكشاف بيانات علمية معقدة (ثلاثية الأبعاد، خرائط جغرافية) بتفاعلية كاملة.
- تضمين رسوم تفاعلية مباشرة في تطبيقات ويب حديثة (React, Streamlit).

١.٣ المميزات والعيوب

الجانب	التفاصيل
مميزات	تفاعلية كاملة افتراضيًا بلا كود إضافي، تصميم عصري جذاب افتراضيًا، تكامل مباشر مع Dash لبناء تطبيقات ويب كاملة، دعم أنواع رسوم متقدمة (خرائط، ثلاثية الأبعاد) بسهولة
عيوب	حجم ملفات HTML الناتجة أكبر بكثير من صور Matplotlib الثابتة، أداء أبطأ مع كميات بيانات ضخمة جدًا داخل رسم واحد، بعض التخصيصات الدقيقة جدًا أقل مرونة من Matplotlib
حدود	غير مناسبة كخيار أول لرسوم مطبوعة أو منشورات أكاديمية تتطلب صيغًا متجهية تقليدية بحتة

١.٤ متى تستخدمها ومتى لا

القرار	الحالة
Plotly (مع Dash)	لوحة معلومات ويب تفاعلية حية
Matplotlib	رسم ثابت للنشر الأكاديمي أو الطباعة
Plotly	استكشاف سريع لبيانات كبيرة مع تكبير وتصفية
Seaborn أو Matplotlib	رسم إحصائي سريع بأقل كود ممكن لتقرير داخلي بسيط

١.٥ أشهر البدائل

- Bokeh: تفاعلية مشابهة، تركيز أكبر على تكامل الويب المباشر بدون خادم.
- Altair: فلسفة تصريحية (Grammar of Graphics)، أبسط للرسوم الإحصائية القياسية.

- Matplotlib + mplcursors: إضافة تفاعلية محدودة فوق Matplotlib التقليدية.

ملخص الفصل

- Plotly وُلدت لحل مشكلة غياب التفاعلية في مكتبات الرسم التقليدية ببايثون.
- قوتها الحقيقية تظهر في لوحات المعلومات الحية عبر Dash؛ لكنها أثقل من Matplotlib للرسوم الثابتة البسيطة.

الفصل الثاني: التثبيت والإعداد

```
pip install plotly
conda install -c conda-forge plotly

# لتصدير رسوم كصور ثابتة (PNG/PDF) وليس HTML فقط
pip install kaleido
```

٢.١ أفضل أسلوب استيراد

```
import plotly.express as px  # الواجهة عالية المستوى (الأكثر استخدامًا)
import plotly.graph_objects as go  # الواجهة منخفضة المستوى (تحكم كامل)
```

٢.٢ بيئات العرض (Rendering)

```
import plotly.io as pio
pio.renderers.default = 'notebook'  # داخل Jupyter
pio.renderers.default = 'browser'  # فتح الرسم في متصفح منفصل من سكربت عادي
```

على عكس Matplotlib، لا تحتاج Plotly عادة إلى plt.show() يدوي داخل Jupyter؛ لكن اختيار الـ renderer المناسب للبيئة (دفتر، متصفح، أو تصدير ملف HTML) هو الخطوة الإعدادية المكافئة له.

أفضل الممارسات

- استخدم plotly.express كنقطة بداية دائمًا؛ انتقل لـ graph_objects فقط عند الحاجة لتحكم غير متاح في express.
- ثبّت kaleido إن احتجت تصدير صور ثابتة (PNG/PDF) بدل ملفات HTML تفاعلية فقط.

الفصل الثالث: فلسفة التصميم

٣.١ واجهتان بفلسفتين مختلفتين

صُممت Plotly بواجهتين متكاملتين لا متنافستين: Plotly Express (px) وهي واجهة تصريحية عالية المستوى مستوحاة من فلسفة Grammar of Graphics (شبيهة بـ ggplot2 في R)، حيث تصف ماذا تريد رسمه من DataFrame واحد بسطر واحد، وتتولى المكتبة بقية التفاصيل. أما Graph Objects (go) فهي واجهة إجرائية منخفضة المستوى تمنحك تحكمًا كاملاً وصريحًا في كل Trace وكل عنصر Layout، مشابهة فلسفيًا للواجهة الكائنية في Matplotlib.

الواجهة	الفلسفة	متى تستخدمها
plotly.express	تصريحية، سطر واحد لكل رسم من DataFrame مرتب	٩٠٪ من الاستخدام اليومي
plotly.graph_objects	إجرائية، تحكم كامل عنصرًا عنصرًا	تخصيصات معقدة أو رسوم مركّبة من مصادر متعددة

٣.٢ البنية الداخلية: Figure = Data + Layout

كل رسم في Plotly هو كائن Figure يتكون من جزأين منفصلين تمامًا: data وهي قائمة من Traces (كل Trace يمثّل مجموعة بيانات مرسومة، مثل خط أو مجموعة أعمدة)، وlayout وهو وصف كل ما هو ليس بيانات (العناوين، الألوان الخلفية، المحاور، حجم الرسم). هذا الفصل الصريح بين البيانات والمظهر مشابه فلسفيًا لفصل NumPy بين البيانات والـ Metadata، ويجعل تعديل مظهر الرسم دون لمس بياناته (أو العكس) عملية طبيعية ومباشرة.

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Scatter(x=[1,2,3], y=[4,5,6], mode='lines+markers')],
    layout=go.Layout(title='مثال'),
)
print(fig.data)     # tuple من الـ Traces
print(fig.layout)   # كائن التخطيط
```

٣.٣ JSON كلغة وصف مشتركة

خلف الكواليس، كل Figure في Plotly ليس أكثر من هيكل بيانات يمكن تمثيله بالكامل كـ JSON، وهذا هو ما يُرسَل فعليًا إلى محرك الرسم plotly.js في المتصفح ليرسمه تفاعليًا. هذا القرار التصميمي يفسّر لماذا يمكن حفظ أي Figure كملف HTML مستقل يعمل بلا حاجة لتثبيت بايثون على الجهاز الذي يفتحه.

```
fig.to_json()          # تمثيل JSON كامل للرسم
fig.write_html('chart.html')   # ملف HTML مستقل تفاعلي بالكامل
```

٣.٤ Templates: فلسفة الأنماط الموحدة

```
import plotly.io as pio
pio.templates.default = 'plotly_dark'
```

Templates في Plotly تلعب دورًا مشابهًا لـ Style Sheets في Matplotlib: تغيير شامل ومتسق للمظهر (الألوان، الخلفية، الخطوط) بسطر واحد يُطبَّق على كل الرسوم اللاحقة في الجلسة.

ملخص الفصل

- واجهتان: express التصريحية السريعة، وgraph_objects الإجرائية للتحكم الكامل.
- كل Figure يتكون من data (قائمة Traces) وlayout (المظهر)، وكلاهما قابل للتمثيل الكامل كـ JSON.
- Templates توفّر تناسقًا بصريًا شاملاً بسطر واحد، مشابهة لـ Style Sheets في Matplotlib.

الفصل الرابع: المفاهيم الأساسية

٤.١ Traces

الـ Trace هو الوحدة الأساسية للبيانات المرسومة، ولكل نوع رسم Trace خاص به (Scatter, Bar, Box, Heatmap...). يمكن لرسم واحد أن يحتوي عدة Traces من أنواع مختلفة متراكبة فوق بعضها.

```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y1, mode='lines', name='المبيعات'))
fig.add_trace(go.Bar(x=x, y=y2, name='الهدف'))
```

٤.٢ Layout

```
fig.update_layout(
    title='لوحة الأداء',
    xaxis_title='الشهر',
    yaxis_title='القيمة',
    template='plotly_white',
    hovermode='x unified',
)
```

٤.٣ التفاعلية المدمجة افتراضيًا

- Hover: عرض تفاصيل القيمة الدقيقة عند تمرير المؤشر فوق أي نقطة.
- Zoom / Pan: تكبير منطقة معينة بالسحب، وتحريك العرض.
- Legend Toggle: النقر على عنصر في legend يُخفي/يُظهر ذلك الـ Trace فورًا.
- Range Slider: شريط تمرير لتحديد نطاق زمني تفاعليًا (شائع في السلاسل الزمنية).

٤.٤ Subplots في Plotly

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2, subplot_titles=('التكلفة', 'المبيعات'))
fig.add_trace(go.Scatter(x=x, y=y1), row=1, col=1)
fig.add_trace(go.Bar(x=x, y=y2), row=1, col=2)
```

الأخطاء الشائعة

- محاولة استخدام معاملات plt من Matplotlib مباشرة في Plotly رغم اختلاف الواجهة كليًا بين المكتبتين.
- بناء رسوم فرعية عبر إنشاء Figures منفصلة بدل make_subplots، مما يفقد التنسيق المشترك بينها.

الفصل الخامس: الكائنات وواجهات البرمجة

٥.١ التوقيع القياسي لدوال Plotly Express

كل دالة رسم في px تتبع نمط توقيع موحد: تمرير DataFrame أولاً، ثم أسماء الأعمدة كنصوص للمحاور والألوان والحجم، مما يجعل التعلم عبر أنواع الرسوم المختلفة سريعًا جدًا بمجرد إتقان النمط مرة واحدة.

```
import plotly.express as px

px.scatter(df, x='age', y='salary', color='department', size='experience')
px.line(df, x='date', y='revenue', color='region')
px.bar(df, x='category', y='sales', color='category')
```

٥.٢ Method Chaining عبر update_*

بعد إنشاء Figure عبر express، الأسلوب الاصطلاحي لتخصيصها أكثر هو استدعاء طرق update_layout وupdate_traces وupdate_xaxes المتسلسلة، والتي تُرجع Figure نفسه، مما يسمح بالتسلسل الكامل (Method Chaining) بعكس محدودية هذا الأسلوب في Matplotlib.

```
fig = (
    px.scatter(df, x='age', y='salary')
    .update_traces(marker=dict(size=10, opacity=0.7))
    .update_layout(title='الراتب مقابل العمر', template='plotly_white')
)
fig.show()
```

٥.٣ نموذج التنفيذ: بناء تصريحي ثم تصيير في المتصفح

عند استدعاء fig.show()، لا تُنفَّذ عملية 'رسم' في بايثون نفسها كما في Matplotlib؛ بل يُبنى هيكل JSON الكامل للرسم، ثم يُرسَل إلى محرك plotly.js (سواء داخل خلية Jupyter أو نافذة متصفح) الذي يتولى الرسم الفعلي والتفاعلية بالكامل. هذا يعني أن بايثون هنا تلعب دور 'مُنشئ الوصف' بينما التنفيذ البصري يحدث خارجها تمامًا في جافاسكريبت.

٥.٤ التكامل مع Dash

```
from dash import Dash, dcc, html

app = Dash(__name__)
app.layout = html.Div([dcc.Graph(figure=fig)])

if __name__ == '__main__':
    app.run(debug=True)
```

أي كائن Figure من Plotly يمكن تمريره مباشرة إلى dcc.Graph في Dash دون أي تحويل، وهذا التكامل السلس هو أحد أهم أسباب انتشار Plotly في بناء لوحات معلومات ويب كاملة.

أفضل الممارسات

- ابدأ دائمًا بـ plotly.express، وانتقل لـ graph_objects فقط عند الحاجة الفعلية لتحكم أدق.

- استخدم update_layout وupdate_traces بأسلوب Method Chaining بدل تعديل fig.layout مباشرة حقلاً حقلاً.

الفصل السادس: أهم الدوال

٦.١ أهم دوال Plotly Express

الدالة	الاستخدام
px.scatter	نقاط تشتت، مع دعم color وsize وanimation_frame
px.line	خطوط، مثالية للسلاسل الزمنية
px.bar	أعمدة، مع دعم barmode='group' أو 'stack'
px.histogram	مدرّج تكراري
px.box / px.violin	توزيعات إحصائية مع كشف القيم الشاذة
px.pie / px.sunburst	نسب دائرية، وsunburst للتسلسلات الهرمية
px.choropleth	خرائط جغرافية ملونة حسب قيمة
px.imshow	خرائط حرارية وصور

```
px.bar(df, x='category', y='sales', color='region', barmode='group')
px.box(df, x='department', y='salary', points='all')  # عرض كل النقاط فوق الصندوق
```

٦.٢ الرسوم المتحركة عبر animation_frame

```
px.scatter(
    df, x='gdp', y='life_exp', size='population', color='continent',
    animation_frame='year', animation_group='country',
    range_x=[0, 100000], range_y=[20, 90],
)
```

animation_frame هي إحدى أقوى ميزات Plotly: تحويل رسم ثابت إلى رسم متحرك تفاعلي عبر بُعد إضافي (غالبًا الزمن) بمعامل واحد فقط، دون أي كود إضافي لإدارة الإطارات كما في FuncAnimation في Matplotlib.

٦.٣ التصدير

```
fig.write_html('report.html')            # تفاعلي كامل
fig.write_image('report.png', scale=2)   # صورة ثابتة (يتطلب kaleido)
fig.write_image('report.pdf')
```

٦.٤ مقارنة بين Plotly وMatplotlib وSeaborn

المعيار	Matplotlib	Seaborn	Plotly
التفاعلية	محدودة/تتطلب إضافات	لا (تعتمد على Matplotlib)	كاملة افتراضيًا
سرعة الرسم الإحصائي	متوسطة (كود أطول)	سريعة جدًا (سطر واحد)	سريعة (سطر واحد عبر express)
جودة النشر الأكاديمي/المطبوع	ممتازة (المعيار الذهبي)	جيدة جدًا	جيدة لكن أقل شيوعًا أكاديميًا
التطبيقات التفاعلية على الويب	غير مناسبة مباشرة	غير مناسبة مباشرة	الخيار الطبيعي (مع Dash)

المعيار	Matplotlib	Seaborn	Plotly
حجم الملف الناتج	صغير (صورة)	صغير (صورة)	أكبر بكثير (HTML/JS)

القاعدة العملية: استخدم Seaborn للاستكشاف الإحصائي السريع بأقل كود، Matplotlib للرسوم الثابتة عالية الدقة للنشر، وPlotly عندما تكون التفاعلية أو النشر على الويب جزءًا أساسيًا من الهدف.

الأخطاء الشائعة

- محاولة تصدير صورة ثابتة عبر write_image دون تثبيت kaleido أولاً.
- استخدام px لرسم بيانات ضخمة جدًا (ملايين النقاط) دون تخفيض العيّنة، مما يجعل ملف HTML الناتج ثقيلاً وبطيء الفتح.

الأسئلة الشائعة

س: هل Plotly مجانية بالكامل للاستخدام في المشاريع الشخصية والتجارية؟

ج: نعم، مكتبة plotly.py مفتوحة المصدر ومجانية بالكامل؛ الخدمات السحابية الإضافية من شركة Plotly (مثل Chart Studio Enterprise) منفصلة وقد تكون مدفوعة.

س: هل يمكن استخدام Plotly بدون Jupyter أو خادم ويب؟

ج: نعم، عبر fig.write_html() ينتج ملف HTML مستقل يعمل بفتحه مباشرة في أي متصفح، أو fig.show() الذي يفتح متصفحًا تلقائيًا من سكربت عادي.

تمارين عملية

١. ارسم رسم تشتت تفاعلي عبر px.scatter لبيانات موظفين (العمر مقابل الراتب) مع تلوين حسب القسم وحجم النقاط حسب سنوات الخبرة.

٢. أنشئ رسمًا متحركًا عبر animation_frame يوضّح تطور مبيعات عدة مناطق عبر الأشهر.

٣. صدّر أي رسم سابق كملف HTML مستقل وكصورة PNG.

حلول مقترحة (تلميحات)

١. px.scatter(df, x='age', y='salary', color='department', size='experience')

٢. px.bar(df, x='region', y='sales', animation_frame='month', range_y=[0, df['sales'].max()])

٣. fig.write_html('out.html'); fig.write_image('out.png')

مراجع وروابط رسمية

- التوثيق الرسمي: https://plotly.com/python/
- مرجع Plotly Express: https://plotly.com/python-api-reference/plotly.express.html

- الكود المصدري: https://github.com/plotly/plotly.py

خاتمة الجزء الأول من Plotly

غطّينا فلسفة Plotly القائمة على data وlayout، وواجهتيها express وgraph_objects، وأهم دوال الرسم والمقارنة مع Matplotlib وSeaborn. الجزء الثاني سيتناول أمثلة عملية متقدمة، الأداء، التصحيح، أفضل الممارسات، ومشاريع مصغّرة.